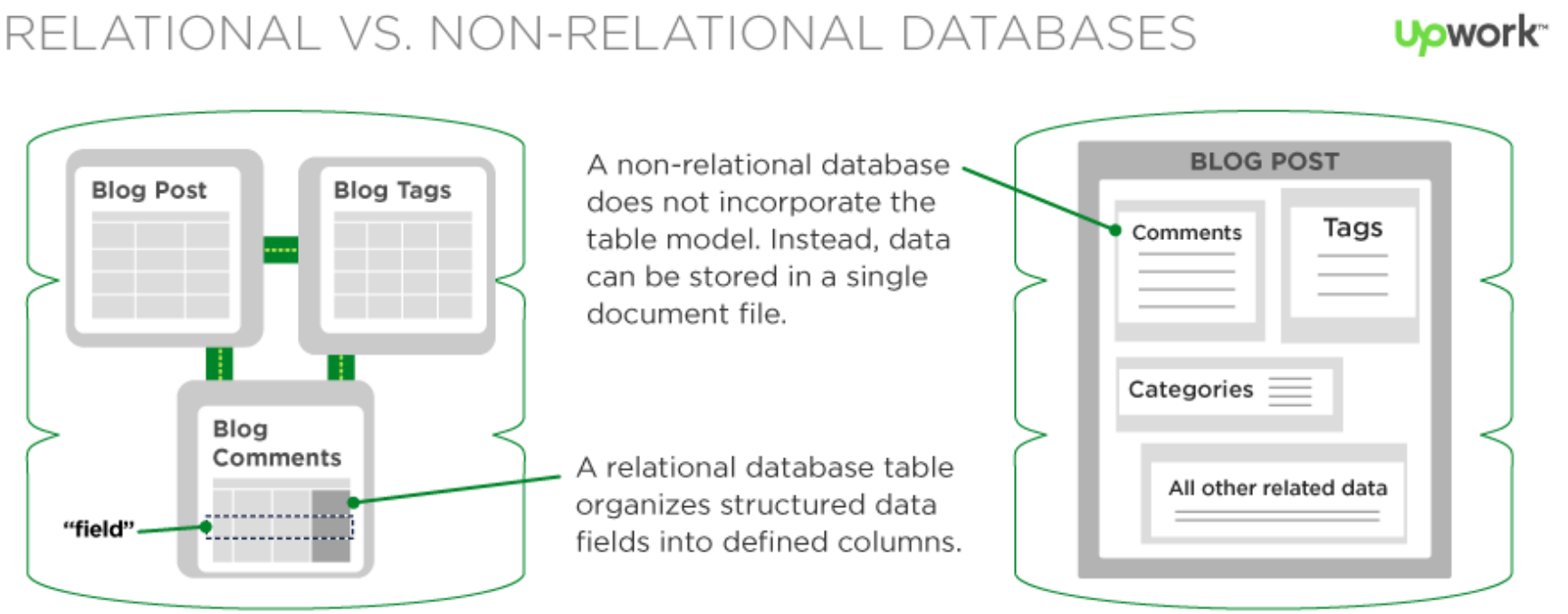
DATABASER
SQL och NoSQL

Olika typer av databaser

- SQL
  - MS SQL Server
  - MySQL
  - Oracle
- NoSQL
  - MongoDB
  - HBase (Hadoop)
  - Apache’s Cassandra DB (Facebook)
- Minnesdatabaser
  - SAP Hana
  - Redis

  - Läs mer i: X4 - SQL vs. NoSQL - What's the difference



Visuell skillnad av lagring av information i SQL och NoSQL

SQL och transaktion – Vad är det?

- En transaktion är en dataförändring i en eller flera delar av databasen/filen.
  - Jämför med att ni ändrar några ord i en eller flera Word-dokument (tex. Sök och ersätt) och sparar dem igen.
- Databasen har bara kvar det senaste tillståndet.
  - Ingen versionshantering.
- Om man förändrar flera ord (sök och ersätt) så måste databasen veta när alla förändringar är klarar och om de har lyckats för att välja att gå vidare eller kunna göra en ”undo” på <u>alla</u> förändringar. Detta för att alla förändringar hänger ihop på något sätt.
  - Hela förändringsjobbet (”ordern”) sparas i en logfil.

SPRÅKET

SQL

```
mysql> select user();
+----------------+
| user()         |
+----------------+
| root@localhost |
+----------------+
1 row in set (0.00 sec)

mysql> select user from mysql.user;
+--------+
| user   |
+--------+
| Konrad |
| root   |
| root   |
| root   |
| test   |
+--------+
5 rows in set (0.30 sec)
```

Spåket SQL

Finns i standarder och med varianter

Vanligast är SQL 1.0

Alla tillverkare har egna varianter på funktionerna som inte ingår i v1.0

Språket MS SQL / MySQL

SQL Statement:

```
SELECT * FROM Customers;
```

Run SQL »

Result:

Number of Records: 91

CustomerID	CustomerName	ContactName	Address
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312
4	Around the Horn	Thomas Hardy	120 Hanover Sq.
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8
6	Blauer See Delikatessen	Hanna Moos	Forsterstr. 57
7	Blondel père et fils	Frédérique Citeaux	24, place Kléber
8	Bólido Comidas preparadas	Martín Sommer	C/ Araquil, 67

http://www.w3schools.com/sql/

http://www.w3schools.com/sql/trysql.asp?filename=trysql_select_all

ÖVRIGA DATABASER

MySQL

SQL databas

Inbyggd Minnesdatabas

- InnoDB

MongoDB

Json representation

Har index

- http://docs.mongodb.org/manual/tutorial/create-an-index/

MongoDB Språket

http://try.mongodb.org/

A MongoDB Shell in your browser Just enoug

```
This shell is a (limited) javascript interpreter, so any commands you are
Try out:
  var a = 5;
  a * 10;
  for(i=0; i<10; i++) { print('hello'); };
You can move onto the next step anytime by typing `next`
> next
MongoDB is a document database. This means that we store data as documents
Here below are a few sample JS objects:
  var a = {age: 25};
  var n = {name: 'Ed', languages: ['c', 'ruby', 'js']};
  var student = {name: 'Jim', scores: [75, 99, 87.2]};
Go ahead and create some documents, then enter 'next'
> next
Here's how you save a document to MongoDB:
  db.scores.save({a: 99});
This says, "save the document '{a: 99}' to the 'scores' collection.".
To confirm that it's been saved properly:
db.scores.find();
>
```

Redis

Minnesdatabas

Stöd för Fritextsökning

Om man vill ha Persistent

- Då skriver Redis ner ändringar I minnet till disk schemalagt

Redis Språket

http://try.redis.io/

> NEXT

Redis can be told that a key should only exist for a certain length of time. This is accomplished v

```
SET resource:lock "Redis Demo"
EXPIRE resource:lock 120
```

This causes the key *resource:lock* to be deleted in 120 seconds. You can test how long a key wi

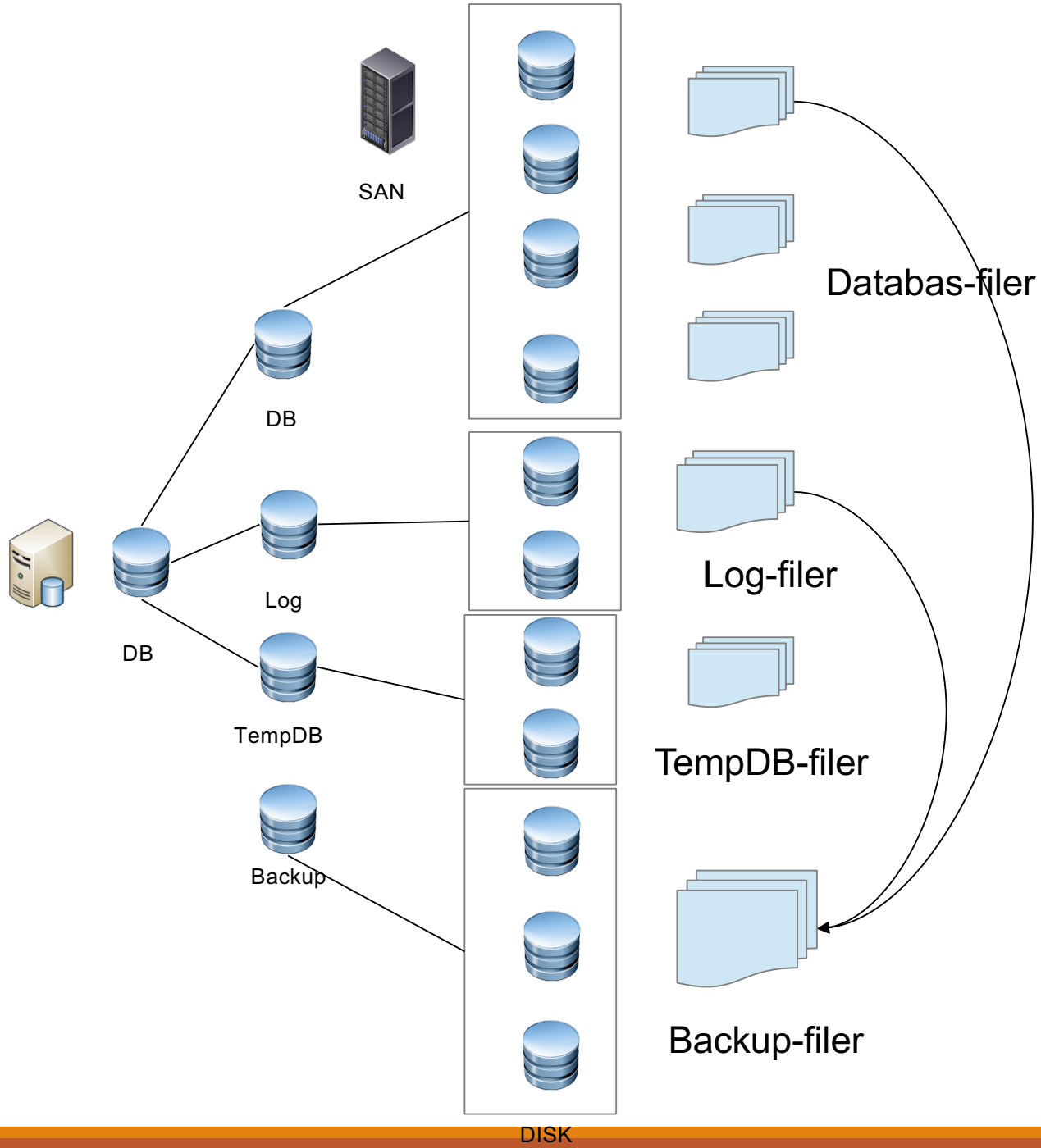
```
TTL resource:lock => 113
// after 113s
TTL resource:lock => -2
```

The *-2* for the `TTL` of the key means that the key does not exist (anymore). A *-1* for the `TTL` of the

```
SET resource:lock "Redis Demo 1"
EXPIRE resource:lock 120
TTL resource:lock => 119
SET resource:lock "Redis Demo 2"
TTL resource:lock => -1
```

DESIGN

GÄLLER MS SQL OCH MYSQL


SAN
DB
Log
DB
TempDB
Backup
DISK
Databas-filer
Log-filer
TempDB-filer
Backup-filer

Rekomendationer för MySQL

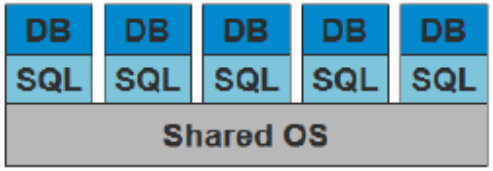
- Storage configuration
- RAID
  - Use a RAID configuration that works well for databases. We used 0+1
  - Use a hardware RAID and not a software-based RAID (e.g. a volume manager)
- HBA Card
  - To optimize the throughput of your HBA card(s), you may want to change their maximum queue depth setting within your ESX configuration.
- In our tests we adjusted the Queue Depth for our QLogic HBA
  - /etc/vmware/esx.conf Set ql2xmaxqdepth=128
- General
  - For optimum performance separate the OS and database disks.
  - Spread your virtual disks across your physical disks
- Be careful not to dynamically reconfigure virtual disks with the MySQL database running. This applies to regular hardware as well, but it's a mistake often made on VMs.

http://www.vmware.com/files/pdf/Virtualization-for-MySQL-on-VMware.pdf

VIRTUALISERING

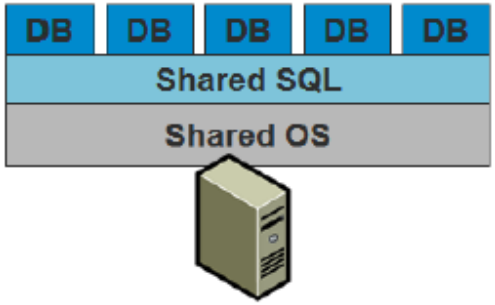
AV MS SQL: INSTANSER

Multi-Instancing



- No OS isolation (configuration, security, fault)
- Resource isolation requires Windows Resource Manager
- No load balancing across physical nodes

Shared Instance

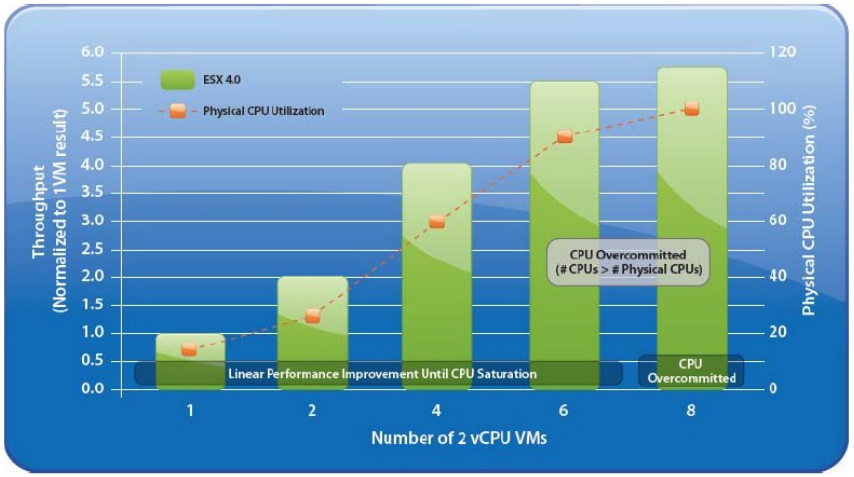


- No OS isolation (configuration, security, fault)
- No Database isolation
- Resource isolation requires SQL Resource Governor
- No load balancing across physical nodes

Virtualisering av MS SQL på egen hårdvara

Prestanda vid konsolidering

Figure 11. Consolidation of Multiple SQL Server Virtual Machines



Each 2 vCPU virtual machine consumes about 15 percent of the total physical CPUs, 5GB of memory in the SQL Server buffer cache, and performs about 3600 I/Os per second (IOPS). As the graph illustrates, the throughput increases linearly as up to four virtual machines (8 vCPUs) are added. As the physical CPUs were overcommitted by increasing the number of virtual machines from four to six (a factor of 1.5), the aggregate throughput increases by a factor of 1.4.

Adding eight virtual machines to this saturates the physical CPUs on this host. ESX 4.0 now schedules 16 vCPUs onto eight physical CPUs, yet the benchmark aggregate throughput increases a further 5 percent as the ESX scheduler is able to deliver more throughput using the few idle cycles left over in the 6 vCPU configuration. Figure 12 shows the ability of ESX to fairly distribute resources in the 8 vCPU configuration.

BACKUP och prestanda

Backup

- Filen i Operativsystemets Filsystem är låst av processen SQL-Server
- Datat måste exporteras av SQL-serverprocessen till en ny fil.
  - Läs och Skrivoperation som inte bör vara mot samma fysiska disk.
- Det gäller att exporten är synkroniserad med Log-filens sista transaktion.
- SQL-server pausar skrivning till databasfilen ett ögonblick och kopierar logfilen samt skapar en ny logfil
- SQL-server kopierar databasfilen till en ny fil och lite förenklat ”buffrar” alla ytterligare förändringar i logfilen och minnet tills backupen är klar.
- Man kan använda databasen under hela backuptiden.

MS SQL / MySQL Performance

http://www.brentozar.com/archive/2008/09/finding-your-san-bottlenecks-with-sqlio/

http://www.brentozar.com/responder/

https://www.youtube.com/watch?v=vvziPI5OQyE

Felsökning

OCH DOKUMENTATION

DiskSpd (SQLIO)

http://aka.ms/diskspd

https://www.brentozar.com/archive/2015/09/getting-started-with-diskspd/

```
timespan:   1
-------------
duration: 60s
warm up time: 0s
cool down time: 0s
measuring latency
random seed: 0
path: 'O:\temp\test.dat'
    think time: 0ms
    burst size: 0
    software and hardware write cache dis
    performing read test
    block size: 2097152
    using interlocked sequential I/O (str
    number of outstanding I/O operations:
    thread stride size: 0
    threads per file: 56
    using I/O Completion Ports
    IO priority: normal
```

MySQL: The Slow Query Log

https://dev.mysql.com/doc/refman/8.0/en/slow-query-log.html

https://www.liquidweb.com/kb/mysql-performance-identifying-long-queries/

MySQL: The Slow Query Log

Enable Logging of Slow Queries (Slow Query Log) in MySQL Database

One of the main requirements for a fast web server is to has efficient and effective SQL statements or queries that are optimized. Any non-optimal SQL (Structured Query Language) commands or statements that take too long or lengthy time to execute will use up a lot of system resources, causing MySQL database to run slower, and then more and more queries backlogs queuing up, and when connection limit is reached, visitors are been denied or refused connection. In worst case scenario, your web server will go down as well, or continuously underperform. The case is especially true when you are using MyISAM table type which uses table-level locking instead of row-level locking in a high traffic website.

Sometime, a single SQL query may be the cause of all the server's problems. MySQL has built-in functionality to capture slow query log or identify queries that are not optimal and take a long time to finish, which allows you to log all slow running queries which took over defined number of seconds to execute by MySQL database engine to a file. Slow query log is not activated or on by default MySQL installation, thus it is one of the less-used logs.

To enable slow query log, simply add the following line to MySQL configuration file (my.cnf or my.ini), and then restart the MySQL server:

An example:

```
[mysqld]
log-slow-queries = slow.log
long_query_time = 20
log-queries-not-using-indexes
```

http://dev.mysql.com/doc/refman/5.1/en/slow-query-log.html
http://www.mydigitallife.info/enable-logging-of-slow-queries-slow-query-log-in-mysql-database/

FÖRDJUPNING

MS SQL SERVER

What Is Microsoft SQL Server 2008?

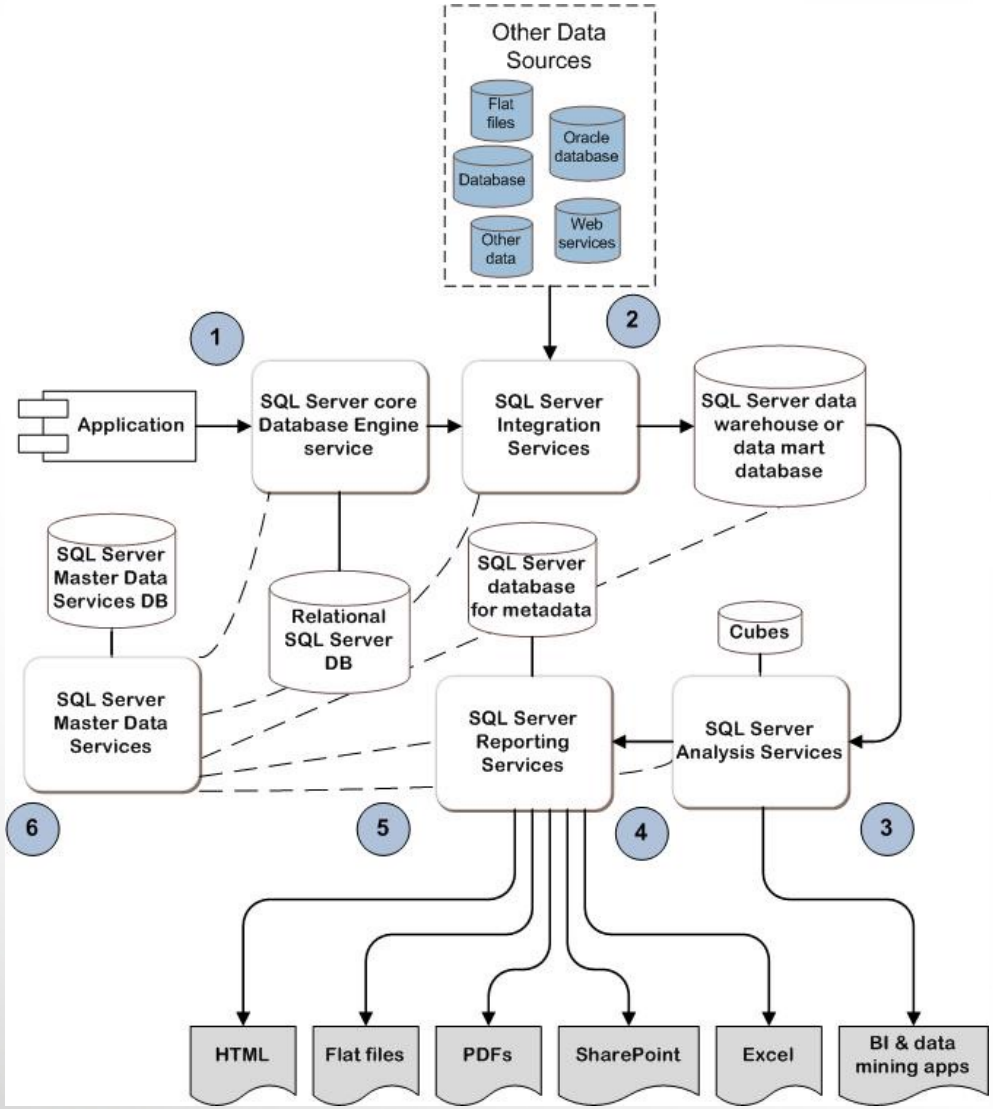
Microsoft SQL Server 2008 is a database solution that includes four primary components:

- Database Engine
- Integration Services
- Analysis Services
- Reporting Services

Microsoft SQL Server 2008 R2 includes a fifth component:

- Master Data Services

Example SQL Server Architecture



Task 4: Determine the Number of Servers

The next task in designing the database infrastructure will be to determine the number of servers required.
Two factors that may change the number of servers running SQL Server are:
Whether scaling out will be implemented.
Whether fault tolerance will be implemented at the database level or instance level.

Scaling Out

Scaling out is the process of distributing a SQL Server load across more than one server; thus the decision to scale out will increase the number of servers required. Multiple scale-out options are available, and this section will provide a brief overview of them. Scale-out options require design changes that occur at the database level and are outside the scope of this guide; therefore, it is expected that any decisions relative to scaling out will have been made in conjunction with the database administrator. Note that a new server, by definition, is a new instance.

Scale-out options include:

Scalable shared databases. With scalable shared databases, a read-only copy of the database on a shared storage area network (SAN) drive is shared between several servers to spread out querying or reporting loads.

Peer-to-peer replication. Instead of several database engines accessing a single copy of the database, multiple copies of the database are made, with each database engine maintaining its own copy of the database. Replication provides scale out, while also allowing updates to the data. Replication is used to propagate changes to all copies of the data.

Linked servers and distributed queries. SQL Server has the ability to query remote database objects as if they were local. This means that a scaled-out database can look like a single large database to an application. The major change to the SQL queries is that the table names will now include the name of the linked server where the data is located.

Distributed partition views. The data in a table is partitioned among tables in several distributed databases, based on a partitioning key. For example, a customer table might be partitioned on customer number ranges: 1–10,000 in one database, 10,001–20,000 in a second database, 20,001–30,000 in a third, and so on. A query that accesses a particular customer number will be executed only on the database that contains the desired customer number. Queries that don't reference a particular customer number will need to be run on all the partitions.

Database-Level Fault Tolerance

Three types of fault tolerance are available to provide protection at the database level:

Database mirroring

Log shipping

Replication

Option 1: Database Mirroring

Database mirroring maintains two copies of a single database that must reside on different server instances (principal and mirror) of SQL Server Database Engine. A principal instance serves the database to clients. The mirror instance acts as a hot or warm standby server, depending on the configuration and state of the mirroring session. When a database mirroring session is synchronized, database mirroring provides a hot standby server that supports rapid failover without a loss of data from committed transactions as long as the applications are configured to support it. When the session is not synchronized, the mirror server is typically available as a warm standby server, with possible data loss.

Database mirroring offers the following benefits:

Uses a single, duplicate copy of the database. If additional copies are required, log shipping can be used on the database in addition to database mirroring.

When database mirroring operates synchronously, provides for zero work loss through delayed commit on the principal database.

Option 2: Log Shipping

Log shipping provides the ability to automatically send transaction log backups from a primary database on a primary server instance to one or more secondary databases on separate secondary server instances. The transaction log backups are applied to each of the secondary databases individually. An optional third server instance, known as the monitor server, records the history and status of backup and restore operations. Optionally, the monitor server provides alerts if these operations fail to occur as scheduled.

Log shipping can be a supplement or an alternative to database mirroring. Although similar in concept, asynchronous database mirroring and log shipping have key differences. Log shipping offers the following distinct capabilities:

Supports multiple secondary databases on multiple server instances for a single primary database.

Allows a user-specified delay during the time when the primary server backs up the log of the primary database and when the secondary servers must restore the log backup. A longer delay can be useful, for example, if data is accidentally changed on the primary database. If the accidental change is noticed quickly, a delay will allow retrieval of still unchanged data from a secondary database before the change is reflected there.

Asynchronous database mirroring has the potential advantage over log shipping in that there is a shorter time between when a given change is made in the primary database and when that change is reflected to the mirror database.

An advantage of database mirroring over log shipping is that high-safety mode is a no-data-loss configuration that is supported as a simple failover strategy.

Option 3: Replication

Replication uses a publish-subscribe model. Using this replication model allows a primary server, referred to as the Publisher, to distribute data to one or more secondary servers, or Subscribers. Replication was not originally built as a fault-tolerant option but can be used as such.

Replication delivers the following benefits:

Allows filtering in the database to provide a subset of data at the secondary databases because it operates at the database level.

Allows more than one redundant copy of the database.

Allows real-time availability and scalability across multiple databases, supporting partitioned updates.

Allows complete availability of the secondary databases for reporting or other functions, without query recovery.

Each of the database-level fault-tolerance options will increase the number of servers running SQL Server in the organization. Determine whether any of the database-level fault-tolerance options will be implemented and, if so, record the number of servers required to support the selected option in Table A-3 in Appendix A.

Instance-Level Fault Tolerance

Failover clustering is the only fault-tolerance option that provides high availability for the instance namespace, which is how applications are configured to connect to the SQL Server database. It provides fault tolerance in the case of a single server failure and supplies these benefits:
Automatic detection and failover
Manual failover
Transparent client redirect

Failover clustering has these constraints:
Operates at the server instance level.
Has no reporting on standby.
Utilizes a single copy of the database.
Does not protect against disk failure.
Does not protect against data corruption.

4.6 Understand Database Consolidation Approaches

Most legacy database applications were traditionally deployed on dedicated physical hardware. Each database instance provisioned onto its own database server. This approach allows the database administration team to provide predictable service levels to their customer base through isolation of those physical resources. Rogue database instances cannot consume resources from other databases and impact their service levels. A significant limitation of this approach is that each of these databases is bounded by the physical resources available on its physical server and consequently, are usually significantly overprovisioned. VMware captured capacity planning data from the servers of our customer base and found that physical database servers use between 5% and 15% of available resources on average (see Figure 14). This is a very inefficient use of server hardware, power and cooling resources, as well as database software licenses. Customers have recognized the inefficiency in this approach and in some cases have looked at physical database consolidation as a solution.

Storage Top 10 Best Practices

1. Understand the IO characteristics of SQL Server and the specific IO requirements / characteristics of your application.

In order to be successful in designing and deploying storage for your SQL Server application, you need to have an understanding of your application's IO characteristics and a basic understanding of SQL Server IO patterns. Performance monitor is the best place to capture this information for an existing application. Some of the questions you should ask yourself here are:

What is the read vs. write ratio of the application?

What are the typical IO rates (IO per second, MB/s & size of the IOs)? Monitor the perfmon counters:

Average read bytes/sec, average write bytes/sec

Reads/sec, writes/sec

Disk read bytes/sec, disk write bytes/sec

Average disk sec/read, average disk sec/write

Average disk queue length

How much IO is sequential in nature, and how much IO is random in nature? Is this primarily an OLTP application or a Relational Data Warehouse application?

To understand the core characteristics of SQL Server IO, refer to SQL Server 2000 I/O Basics.

Storage Top 10 Best Practices

2. **More / faster spindles are better for performance**
Ensure that you have an adequate number of spindles to support your IO requirements with an acceptable latency.
Use filegroups for administration requirements such as backup / restore, partial database availability, etc.
Use data files to “stripe” the database across your specific IO configuration (physical disks, LUNs, etc.).

3. **Try not to “over” optimize the design of the storage; simpler designs generally offer good performance and more flexibility.**
Unless you understand the application very well avoid trying to over optimize the IO by selectively placing objects on separate spindles.
Make sure to give thought to the growth strategy up front. As your data size grows, how will you manage growth of data files / LUNs / RAID groups? It is much better to design for this up front than to rebalance data files or LUN(s) later in a production deployment.

Storage Top 10 Best Practices

4. Validate configurations prior to deployment
Do basic throughput testing of the IO subsystem prior to deploying SQL Server. Make sure these tests are able to achieve your IO requirements with an acceptable latency. SQLIO is one such tool which can be used for this. A document is included with the tool with basics of testing an IO subsystem. Download the SQLIO Disk Subsystem Benchmark Tool.
Understand that the of purpose running the SQLIO tests is not to simulate SQL Server's exact IO characteristics but rather to test maximum throughput achievable by the IO subsystem for common SQL Server IO types.
IOMETER can be used as an alternative to SQLIO.

5. **Always place log files on RAID 1+0 (or RAID 1) disks. This provides:**
better protection from hardware failure, and
better write performance. Note: In general RAID 1+0 will provide better throughput for write-intensive applications. The amount of performance gained will vary based on the HW vendor's RAID implementations. Most common alternative to RAID 1+0 is RAID 5. Generally, RAID 1+0 provides better write performance than any other RAID level providing data protection, including RAID 5.

Storage Top 10 Best Practices

6. Isolate log from data at the physical disk level
When this is not possible (e.g., consolidated SQL environments) consider I/O characteristics and group similar I/O characteristics (i.e. all logs) on common spindles.
Combining heterogeneous workloads (workloads with very different IO and latency characteristics) can have negative effects on overall performance (e.g., placing Exchange and SQL data on the same physical spindles).

7. **Consider configuration of TEMPDB database (What Is This?)**
Make sure to move TEMPDB to adequate storage and pre-size after installing SQL Server.
Performance may benefit if TEMPDB is placed on RAID 1+0 (dependent on TEMPDB usage).
For the TEMPDB database, create 1 data file per CPU, as described in #8 below.

8. **Lining up the number of data files with CPU's has scalability advantages for allocation intensive workloads.**
It is recommended to have .25 to 1 data files (per filegroup) for each CPU on the host server.
This is especially true for TEMPDB where the recommendation is 1 data file per CPU.
Dual core counts as 2 CPUs; **logical procs (hyperthreading) do not.**

TEMPDB

Recommendations for managing TEMPDB

The tempdb files must be configured with initial size and auto-growth based on your workloads. Do not let with the default sizes.
The tempdb files must be located on RAID0 (for better performance) or RAID1 (if you need have more writes than reads) or RAID5 (if you have more reads than writes). RAID10 is the best option but no all companies can justify this.
The tempdb files must be located on separated disks to avoid contention issues and improves the performance.
Tempdb database must be created with one data file per physical processor if the quantity of CPUs is eight or less. If there are more then eight CPUs then you can start off with eight data files and after increase the number of files by four in case there was PAGELATCH contentions on one of the allocation bitmap pages (including PFS pages, GAM pages and SGAM pages) until the issue is solved, if not, add four files more, and so on.
Do not forget, tempdb data files or log file should not grow so much in a short time, if this happens then something is working wrong, so you need to analyze the workloads performance and detect the query that you have to optimize.

Major Problems with TEMPDB:-
1. tempdb has run out of space.
2. tempdb is experiencing I/O bottleneck
3. tempdb is experiencing contention

TEMPDB Forts.

Q: Virtualized OS, SAN, no dedicated LUNs, most likely scenario with no gotchas, theoretically: dump TempDB with everything else all on one virtual volume, including logs — or split to separate virtual volumes and separate from other databases and also from logs? (Matthew N.)

When building virtual SQL Server machines, always use a separate VMDK/VHD for TempDB. This lets your VMware and Hyper-V sysadmins manage your TempDB differently. Some storage systems can automatically move different VHD/VMDK files to different tiers of storage based on their access patterns. If you suddenly enable RCSI and need a lot more TempDB throughput, it's easy for your VMware admin to adjust that when it's in a separate file (or LUN).

http://www.brentozar.com/archive/2013/12/video-the-secrets-of-tempdb/

http://support.microsoft.com/kb/2154845

http://www.brentozar.com/archive/2013/10/frequently-asked-questions-about-tempdb/

Storage Top 10 Best Practices

9. Don't overlook some of SQL Server basics
Data files should be of equal size – SQL Server uses a proportional fill algorithm that favors allocations in files with more free space.
Pre-size data and log files.
Do not rely on AUTOGROW, instead manage the growth of these files manually. You may leave AUTOGROW ON for safety reasons, but you should proactively manage the growth of the data files.

10. Don't overlook storage configuration bases
Use up-to-date HBA drivers recommended by the storage vendor
Utilize storage vendor specific drivers from the HBA manufactures website
Tune HBA driver settings as needed for your IO volumes. In general driver specific settings should come from the storage vendor. However **we have found that Queue Depth defaults are usually not deep enough** to support SQL Server IO volumes.
Ensure that the storage array firmware is up to the latest recommended level.
Use multipath software to achieve balancing across HBA's and LUN's and ensure this is functioning properly
Simplifies configuration & offers advantages for availability
Microsoft Multipath I/O (MPIO): Vendors build Device Specific Modules (DSM) on top of Driver Development Kit provided by Microsoft.

http://technet.microsoft.com/library/Cc966534